

Lab 8 Report: GCD Calculator Using an FSM

Elguja Modebadze

April 29, 2026

1 Introduction

In this lab, a digital circuit was designed to compute the greatest common divisor (GCD) of two 8-bit numbers entered in two's complement format. The design was implemented in SystemVerilog and targeted for FPGA synthesis. Because the assignment required the circuit to operate at a clock frequency of at least 60 MHz, division and remainder operations were avoided. Instead, the Euclidean subtraction algorithm was used, since it is more hardware-friendly and can be implemented efficiently using a finite state machine (FSM).

2 Problem Description

The circuit receives two signed 8-bit numbers. Since only four switches are available for data entry, each number must be entered in two steps: first the lower 4 bits, then the upper 4 bits. This process must be repeated for both input numbers. The values are interpreted in two's complement format, which means the most significant bit represents the sign. For example, the value 11111111 corresponds to -1 .

After both numbers are entered, the circuit computes their GCD and shows the result on the LEDs. The GCD is computed using the absolute values of the two numbers, since the greatest common divisor is taken to be nonnegative.

3 Design Approach

The solution was implemented as a finite state machine. The FSM controls both the input process and the calculation process. Separate states are used to load the lower and upper nibbles of the first input A , then the lower and upper nibbles of the second input B . After both values are captured, the FSM converts them into absolute values and then enters the calculation stage.

The Euclidean subtraction algorithm was chosen for the computation. The main idea of this algorithm is simple: if one value is larger than the other, the larger value is replaced by the difference between the two. This process is repeated until both values become equal, at which point that value is the GCD. The algorithm also handles zero cases naturally. If one value becomes zero, the other value is the GCD.

This method is well suited for hardware because each clock cycle performs only a comparison and, if necessary, one subtraction. This keeps the combinational logic relatively simple and helps the design achieve a high maximum clock frequency.

4 FSM Description

The FSM uses the following states:

- **S_A_LO**: wait for the lower 4 bits of A
- **S_A_HI**: wait for the upper 4 bits of A
- **S_B_LO**: wait for the lower 4 bits of B
- **S_B_HI**: wait for the upper 4 bits of B
- **S_PREP**: convert both inputs to absolute values
- **S_CALC**: perform the Euclidean subtraction algorithm
- **S_DONE**: display the result and wait for a new computation

A simple button lock mechanism is used to ensure that one button press causes only one action. This prevents repeated state transitions while the button is still being held down.

5 Two's Complement Conversion

The two inputs are stored as signed 8-bit values. Before starting the GCD calculation, both numbers are converted to their absolute values. This is done by checking the most significant bit. If the number is negative, its absolute value is obtained by inverting the bits and adding one.

The helper function used for this purpose is shown below.

```
1 function automatic [7:0] abs_tc8(input logic signed [7:0] x);  
2     begin  
3         if (x[7] == 1'b1)  
4             abs_tc8 = ~x + 8'd1;  
5         else  
6             abs_tc8 = x[7:0];  
7     end  
8 endfunction
```

6 Main Calculation Logic

The core of the design is the calculation state, where the Euclidean subtraction algorithm is applied. The logic checks whether one of the values is zero, whether both values are equal, or which of the two is larger. If one value is larger, that value is reduced by subtracting the smaller one.

A simplified excerpt of the calculation logic is shown below.

```
1 S_CALC: begin  
2     if (a_abs == 8'd0) begin  
3         result <= b_abs;  
4         led <= b_abs;
```

```

5         state    <= S_DONE;
6     end
7     else if (b_abs == 8'd0) begin
8         result <= a_abs;
9         led    <= a_abs;
10        state   <= S_DONE;
11    end
12    else if (a_abs == b_abs) begin
13        result <= a_abs;
14        led    <= a_abs;
15        state   <= S_DONE;
16    end
17    else if (a_abs > b_abs) begin
18        a_abs <= a_abs - b_abs;
19    end
20    else begin
21        b_abs <= b_abs - a_abs;
22    end
23 end

```

This state performs one subtraction per clock cycle, which means the circuit computes the result iteratively rather than in a single cycle.

7 Example of Operation

As an example, consider the case $A = 12$ and $B = -68$. The value 12 is entered as 00001100, while -68 is entered in two's complement as 10111100. After input, the circuit converts the numbers into their absolute values, obtaining 12 and 68. The subtraction process then proceeds as follows:

$$68 - 12 = 56, \quad 56 - 12 = 44, \quad 44 - 12 = 32, \quad 32 - 12 = 20, \quad 20 - 12 = 8$$

Now the pair becomes (12, 8), then:

$$12 - 8 = 4, \quad 8 - 4 = 4$$

Since both values are now equal, the GCD is 4. This result is displayed on the LEDs.

8 Finite State Machine

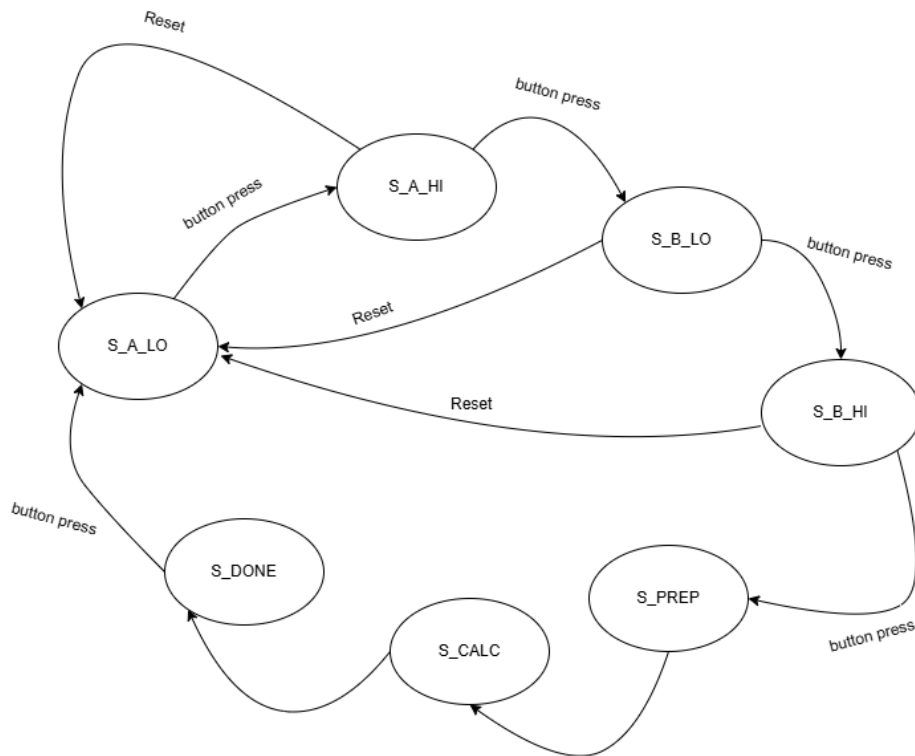


Figure 1: Finite State Machine for GCD computation

9 Timing Requirement

One of the main requirements of the assignment was that the design should be able to operate at at least 60 MHz. The FSM-based subtraction approach is suitable for this because it avoids expensive arithmetic operations such as division and modulo. After synthesis, the design achieved a maximum operating frequency above the required threshold, which confirms that the timing requirement was satisfied.

10 Conclusion

In this lab, a GCD calculator for two signed 8-bit numbers was successfully designed using SystemVerilog and an FSM-based control structure. The circuit accepts values in two's complement format, converts them to absolute values, and computes the GCD using the Euclidean subtraction algorithm. The design avoids division and remainder operations, making it more efficient for FPGA implementation. The FSM cleanly organizes the input process, computation, and result display. Overall, the design satisfies both the functional and timing requirements of the assignment.